CURE CLUSTERING

CURE Algorithm: Clustering Using REpresentatives

Why Do We Need CURE?

- **The Problem with Standard Algorithms (k-Means, BIRCH):**
  - Assume clusters are **spherical or ellipsoidal** in shape.
  - Fail completely on complex, irregular, or non-convex geometries (e.g., concentric circles, elongated paths, S-shapes).
  - Highly sensitive to outliers, which pull centroids away from true cluster centers.

- **The Traditional Solution Trade-off:**
  - **Single Centroid:** Fast, but loses all spatial shape information.
  - **All-Points (Hierarchical Single-Link):** Captures shape, but is extremely slow ($O(N^2)$ or $O(N^3)$) and susceptible to the "chaining effect".

- **The CURE Solution:**
  - Uses **multiple representative points** per cluster instead of a single centroid.
  - **Shrinks** those points inward toward the centroid to dampen outlier effects.

The main idea behind **CURE** (*Clustering Using REpresentatives*) is simple: **don't try to represent a whole cluster with just one single middle point (centroid)**.

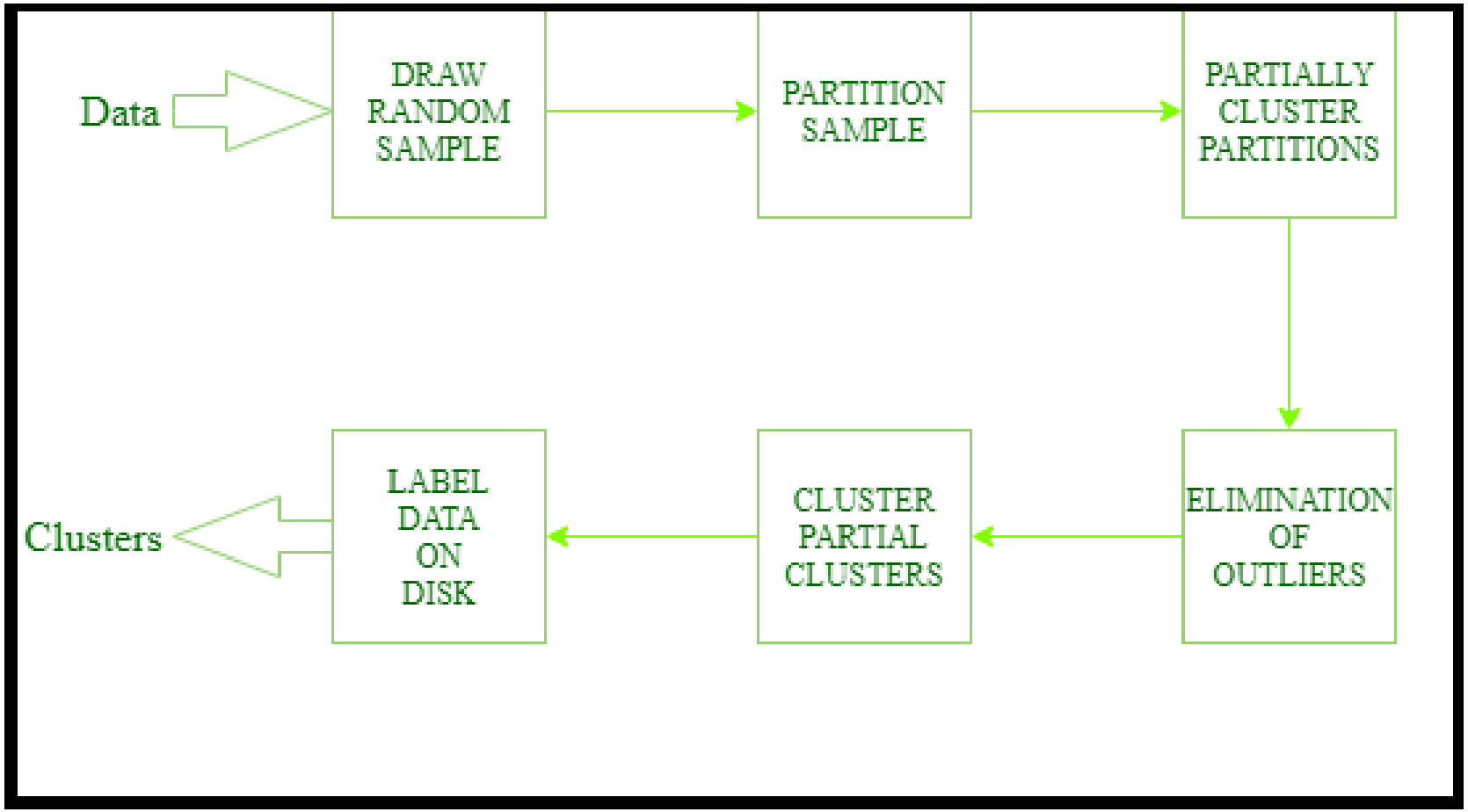
If a cluster looks like a curved banana or a long rectangle, a single centroid in the center fails to capture its actual shape. Instead, CURE picks **multiple scattered points** around the cluster's edges and **shrinks them slightly inward** toward the center.

Simple Visual Analogy

Imagine trying to describe the shape of a country:

- **Centroid approach (k-means):** You place a single pin right in the geographic center. It tells you where the center is, but completely misses whether the country is long, thin, or island-shaped.
- **CURE approach:** You place pins at key boundary towns (north, south, east, west) so you capture the real outline. Then you pull those border pins slightly inward so an isolated island far offshore doesn't distort your map.

CURE Architecture Workflow



How CURE Works

① Sampling & Partitioning

Scale down the data

Draw a random sample from the dataset. If it's still too large for main memory, partition the sample into p smaller blocks and partially cluster each partition locally to speed up execution.

② Pick Representative Points

Capture the shape

Instead of representing a cluster by a single centroid point, select c well-scattered points across the cluster. This preserves the actual geometric boundary/shape of the cluster.

③ Shrink Representatives

Reduce outlier impact

Shrink the chosen representative points toward the cluster centroid by a fraction α (typically $\alpha \approx 0.2$ to 0.7). Shrinking pulls fringe points inward, softening the impact of outliers.

④ Hierarchical Merging

Combine clusters

Merge the two clusters that have the closest pair of representative points. Repeat until the target number of clusters k is reached.

Step-by-Step Example

Let's walk through CURE on a 2D dataset with a banana-shaped cluster A, a circular cluster B, and an outlier point O.

Step 1: Sampling & Pre-clustering

- **Data:** A huge dataset with 1,000,000 points.
- **Action:** Draw a random sample of 2,000 points into memory.
- Divide these 2,000 points into smaller partitions (e.g., 4 partitions of 500 points each) and run initial hierarchical clustering on each partition to form initial mini-clusters.

Step 2: Selecting Representative Points

For a formed mini-cluster (like our banana-shaped Cluster A):

1. Find the first point p_1 furthest from the cluster's center.
2. Find p_2, the point furthest from p_1.
3. Find p_3, the point furthest from both p_1 and p_2.
4. Repeat until you have c scattered points (e.g., $c = 4$).

Result: These 4 points sit at the tip, tail, and top/bottom curves of the banana shape, defining its real geometry.

Step 3: Shrinking the Representatives (α)

To prevent extreme noisy points or outliers on the edges from having too much influence, shrink all 4 representative points toward the cluster centroid by a fraction α (e.g., $\alpha = 0.3$, or 30%).

$$\text{New Point} = \text{Old Point} + \alpha \times (\text{Centroid} - \text{Old Point})$$

Result: The boundary pins pull inward slightly. If point O was an outlier, shrinking pulls it away from external space, reducing its impact during merging.

Step 4: Merging Clusters

To decide which two clusters to merge:

- Calculate the distance between cluster A and cluster B as the **shortest distance between any shrunk representative point of A and any shrunk representative point of B.**
- Merge the pair of clusters with the smallest distance.
- Repeat until you reach the target count k.

Step 5: Assigning the Remaining Points

Finally, stream through the remaining points in the original dataset and assign each point to the cluster with the nearest shrunk representative point.

Why CURE Wins Over *k*-Means

Feature	k-Means	CURE
Shape handling	Spherical / circular blobs only	Arbitrary, non-spherical shapes (curved, elongated)
Outlier sensitivity	High (outliers drag the centroid off-center)	Low (shrinking α dampens outlier effects)
Memory usage	Needs many passes over full dataset	Runs on a small, smart sample in memory

CURE Example:

Suppose we have a cluster C_1 containing 4 points in 2D space:

$$P_1 = (1,1), \quad P_2 = (1,5), \quad P_3 = (5,1), \quad P_4 = (5,5)$$

1. Pick $c = 2$ representative points for C_1.
2. Shrink these representatives toward the centroid using shrinking factor $\alpha = 0.5$ (50%).
3. Find the distance between C_1 and another cluster C_2.

Step 1: Find the Cluster Centroid (μ_1)

The centroid is the mean of all points in C_1:

$$\mu_1 = \left(\frac{1+1+5+5}{4}, \frac{1+5+1+5}{4} \right) = (3, 3)$$

Step 2: Select $c = 2$ Representative Points

1. First representative point (r_1):

Pick the point in C_1 furthest from the centroid $\mu_1 = (3, 3)$:

- $d(P_1, \mu_1) = \sqrt{(1-3)^2 + (1-3)^2} = \sqrt{4+4} = \sqrt{8} \approx 2.83$
- All four points (P_1, P_2, P_3, P_4) are at distance ≈ 2.83 from the center.

We select $r_1 = P_1 = (1, 1)$.

2. Second representative point (r_2):

Pick the point in C_1 furthest from $r_1 = (1, 1)$:

- $d(P_2, r_1) = \sqrt{(1-1)^2 + (5-1)^2} = 4.0$
- $d(P_3, r_1) = \sqrt{(5-1)^2 + (1-1)^2} = 4.0$
- $d(P_4, r_1) = \sqrt{(5-1)^2 + (5-1)^2} = \sqrt{16+16} = \sqrt{32} \approx 5.66$

The maximum distance is to P_4. So we select $r_2 = P_4 = (5, 5)$.

Selected Representatives for C_1: $\{(1, 1), (5, 5)\}$

Step 3: Shrink Representatives Toward Centroid ($\alpha = 0.5$)

Formula for shrinking a point r:

$$r' = r + \alpha \cdot (\mu_1 - r)$$

Shrinking $r_1 = (1, 1)$:

$$r_1' = (1, 1) + 0.5 \cdot ((3, 3) - (1, 1))$$

$$r_1' = (1, 1) + 0.5 \cdot (2, 2) = (1, 1) + (1, 1) = (\mathbf{2}, \mathbf{2})$$

Shrinking $r_2 = (5, 5)$:

$$r_2' = (5, 5) + 0.5 \cdot ((3, 3) - (5, 5))$$

$$r_2' = (5, 5) + 0.5 \cdot (-2, -2) = (5, 5) + (-1, -1) = (\mathbf{4}, \mathbf{4})$$

Shrunk Representatives for C_1: $S_1 = \{(2, 2), (4, 4)\}$

Step 4: Calculate Distance Between Clusters

Suppose another cluster C_2 has shrunk representative points $S_2 = \{(10, 2), (10, 4)\}$.

The distance between C_1 and C_2 in CURE is the **minimum distance between any pair of shrunk representative points**:

$$d(C_1, C_2) = \min_{u \in S_1, v \in S_2} d(u, v)$$

Pair (u,v)	Distance Calculation	Result
$(2, 2)$ to $(10, 2)$	$\sqrt{(10-2)^2 + (2-2)^2} = \sqrt{64}$	8.0
$(2, 2)$ to $(10, 4)$	$\sqrt{(10-2)^2 + (4-2)^2} = \sqrt{64+4}$	8.25
$(4, 4)$ to $(10, 2)$	$\sqrt{(10-4)^2 + (2-4)^2} = \sqrt{36+4}$	6.32
$(4, 4)$ to $(10, 4)$	$\sqrt{(10-4)^2 + (4-4)^2} = \sqrt{36}$	6.0

Final Result:

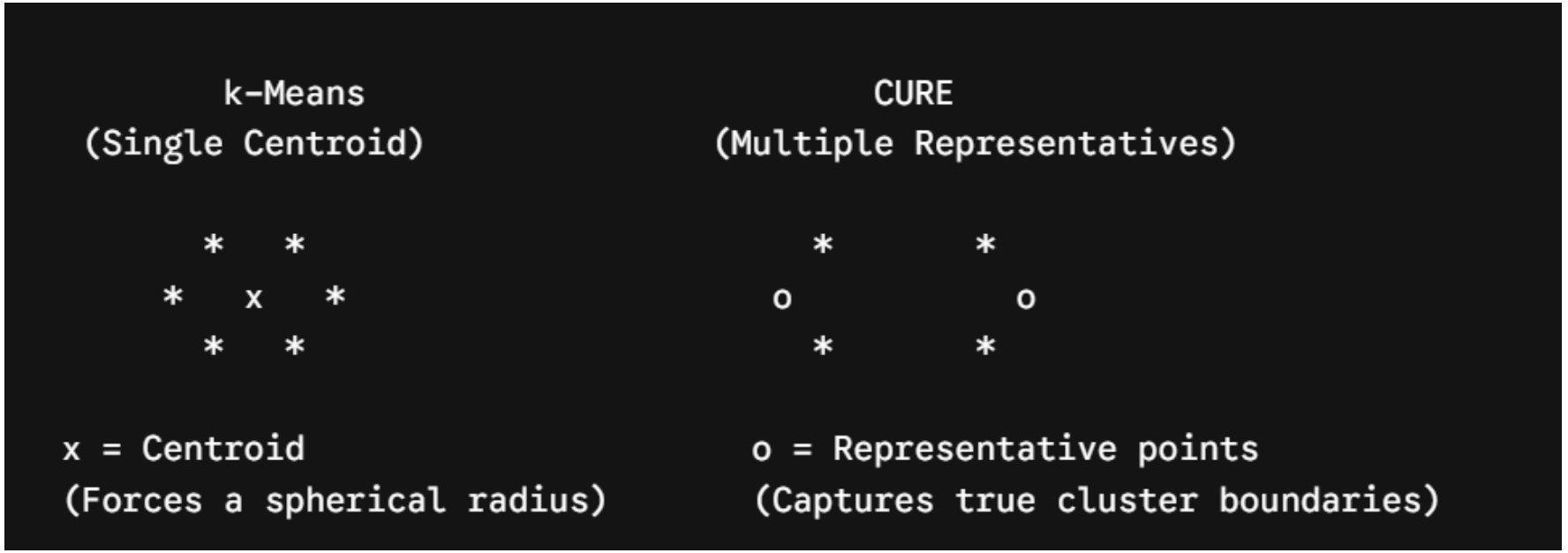
$$d(C_1, C_2) = 6.0$$

Since 6.0 is the minimum distance among all representative pairs, CURE uses 6.0 as the distance between C_1 and C_2 when deciding whether to merge them.

Core Structural Differences

Feature	k-Means	CURE
Cluster Representation	Represents a cluster by a single point (the centroid / mean).	Represents a cluster by multiple scattered points that are shrunk inward toward the center.
Cluster Shape Capability	Only finds spherical/circular blobs. Fails on elongated or curved shapes.	Finds arbitrary shapes (non-spherical, curved, concentric rings, elongated clusters).
Outlier Sensitivity	High. Outliers pull the centroid away, distorting cluster boundaries.	Low. Shrinking representative points toward the center reduces the pull of extreme noise.
Category of Algorithm	Partitioning algorithm (iterative assignment to nearest center).	Hierarchical algorithm (bottom-up merging based on representative distance).
Distance Metric Requirement	Requires a space where a mean can be computed (Euclidean space).	Works with any distance metric that can measure point-to-point proximity.
Scalability Method	Iterates over the entire dataset repeatedly until convergence.	Samples and partitions large datasets so it can run efficiently in memory.

Visual & Conceptual Difference



- **In k-means:** Because every point is assigned to its *closest single centroid*, boundaries between clusters are always straight lines (Voronoi cells). This makes it impossible to separate two interlocking curved clusters.
- **In CURE:** Because distance is measured to the *nearest shrunk representative point*, the boundary can curve and bend naturally along the actual shape of the data.

When to Use Which?

Use k-means when:

- You have basic numeric data and expect well-separated, spherical clusters.
- You need a fast, simple algorithm with straightforward implementation.

Use CURE when:

- Your data has complex geometries (e.g., rivers, roads, crescent shapes).
- Your dataset contains significant noise or outliers.
- The dataset is too large to fit into memory, so random sampling and partitioning are required.

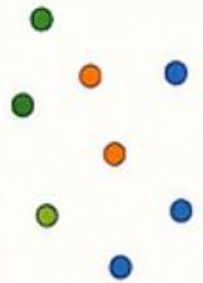
CURE (Clustering Using REpresentatives)

Why do we need CURE?

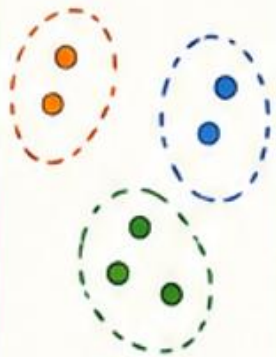
- k-means uses only centroid → Sensitive to outliers and assumes spherical clusters.
- Agglomerative merging → One bad merge cannot be undone.
- CURE represents a cluster by multiple representative points (not centroid).
- Representative points are shrunk towards the centroid to avoid boundary problems.
- Robust to outliers and can handle arbitrarily shaped clusters.

Core Idea of CURE

1. Initially, each point is a cluster.



2. Merge nearest clusters.



3. For each cluster: Find centroid and choose c representative points (farthest from centroid).



4. Shrink representative points towards centroid by shrink factor α.



Distance between clusters = minimum distance between any two representative points.

DATASET (Running Example)

Point	Coordinates (x, y)
A	(1, 2)
B	(2, 1)
C	(2, 3)
D	(8, 8)
E	(9, 9)
F	(8, 10)

We want : 2 clusters

Representative points (c) = 2

Shrink factor (α) = 0.2

Distance Metric : Euclidean

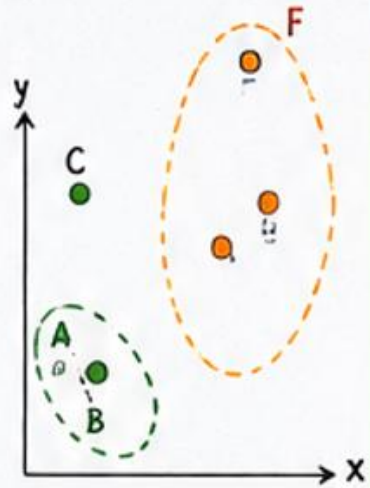
1 INITIAL DISTANCE MATRIX

	A	B	C	D	E	F
A	–	1.41	1.41	9.22	11.31	11.18
B	1.41	–	2.00	9.22	10.63	10.30
C	1.41	2.00	–	8.60	10.00	9.43
D	9.22	9.22	8.60	–	1.41	2.00
E	11.31	10.63	10.00	1.41	–	1.41
F	11.18	10.30	9.43	2.00	1.41	–

(Distances are rounded to 2 decimal places)

2 FIRST MERGES (Nearest Clusters)

- Smallest distance = 1.41
  → (A, B) merge
- Next smallest = 1.41
  → (D, E) merge



3 CENTROIDS OF MERGED CLUSTERS

Cluster AB:

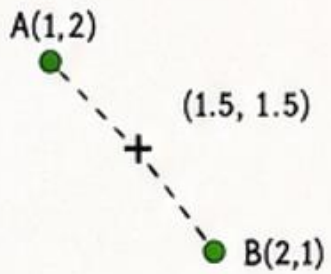
$$C_{AB} = \left(\frac{1+2}{2}, \frac{2+1}{2}\right) = (1.5, 1.5)$$

Cluster DE:

$$C_{DE} = \left(\frac{8+9}{2}, \frac{8+9}{2}\right) = (8.5, 8.5)$$

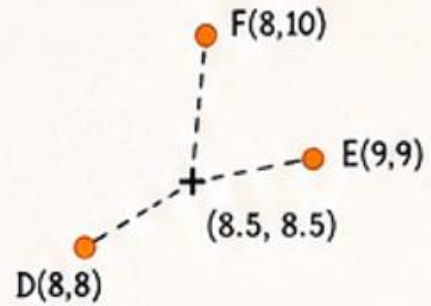
4 SELECT REPRESENTATIVE POINTS (c = 2)

Cluster AB (Centroid = (1.5, 1.5))



Representative points: A, B

Cluster DE (Centroid = (8.5, 8.5))



Representative points: D, E

5 SHRINK REPRESENTATIVE POINTS TOWARDS CENTROID

Formula: $R' = R + \alpha (C - R)$ $(\alpha = 0.2)$

Cluster AB (C = (1.5, 1.5))

A(1,2):

$A' = (1,2) + 0.2[(1.5, 1.5) - (1,2)]$
$= (1,2) + 0.2(0.5, -0.5)$
$= (1.1, 1.9)$

B(2,1):

$B' = (2,1) + 0.2[(1.5, 1.5) - (2,1)]$
$= (2,1) + 0.2(-0.5, 0.5)$
$= (1.9, 1.1)$

Cluster DE (C = (8.5, 8.5))

D(8,8):

$D' = (8,8) + 0.2[(8.5, 8.5) - (8,8)]$
$= (8,8) + 0.2(0.5, 0.5)$
$= (8.1, 8.1)$

E(9,9):

$E' = (9,9) + 0.2[(8.5, 8.5) - (9,9)]$
$= (9,9) + 0.2(-0.5, -0.5)$
$= (8.9, 8.9)$

6 DISTANCE BETWEEN CLUSTERS
(Use shrunk representative points)

AB representatives: A′(1.1, 1.9), B′(1.9, 1.1)
DE representatives: D′(8.1, 8.1), E′(8.9, 8.9)

Pair	Distance
A′ – D′	9.90
A′ – E′	11.03
B′ – D′	8.73
B′ – E′	8.73

↳ Cluster distance = minimum distance between any pair of representatives.

7 NEXT MERGE

Minimum distance = 8.73
→ Merge Cluster AB with C
(Compute distance from C to representatives of AB)

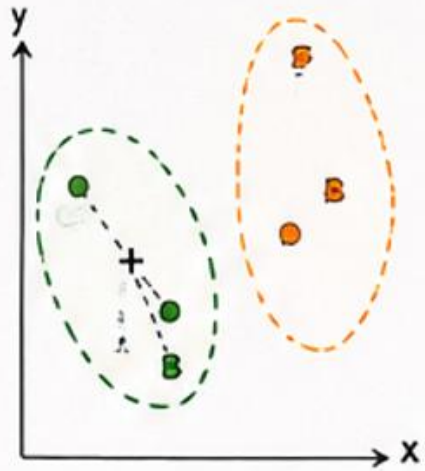
Distances:
C(2,3) – A′(1.1, 1.9) = 1.49
C(2,3) – B′(1.9, 1.1) = 1.90
Minimum = 1.49

So, merge C with Cluster AB.

8 REPEAT (Update, Shrink, Compute Distance)

Form new cluster ABC
- Centroid C_{ABC} = (1.67, 2.00)
- Choose 2 representative points (farthest from centroid): A and C
- Shrink A and C towards C_{ABC} ($\alpha = 0.2$)
- Nove compare ABC with DE using representative points
- Merge them (to get 2 clusters)

Final Clusters: {A, B, C} and {D, E, F}



ALGORITHM (CURE)

1. Each data point is a cluster.
2. Compute the closest pair of clusters using representative points.
3. Merge the pair of clusters.
4. Compute centroid of the new cluster.
5. Select c representative points from the cluster (farthest from centroid).
6. Shrink the representative points towards centroid using shrink factor α.
7. Repeat steps 2–6 until desired number of clusters are obtained.

COMPLEXITY

Let n = number of points,
c = number of representative points per cluster.

- Distance computation: $O(n^2)$
- Total time complexity: $O(n^2 \log n)$ (approx.)
- Space complexity: $O(n)$

ADVANTAGES

- ✓ Handles arbitrarily shaped clusters.
- ✓ Robust to outliers.
- ✓ Not sensitive to noise.
- ✓ Good for large datasets (sampling can be used).
- ✓ More accurate than k-means for complex clusters.

COMPARISON

Feature	k-Means	Agglomerative	CURE
Cluster Shape	Spherical	Any	Any
Outlier Handling	Poor	Moderate	Excellent
Centroid Based	Yes	No	No
Representative Points	No	No	Yes
Scalability	High	Low	Good
Complexity	Low	High	Moderate
Parameters	k	Linkage	c, α

Key Takeaway: CURE uses multiple representative points and shrinkage to capture the shape of clusters and resist the effect of outliers.

BFR Algorithm

The **BFR algorithm** (named after its creators **Bradley, Fayyad, and Reina**) is a specialized variant of k-means designed to process massive, multi-gigabyte datasets that are **too large to fit into main memory (RAM)**.

While standard k-means requires reading the entire dataset from disk into memory across dozens of iterations, BFR achieves massive scalability by reading the data from disk in **chunks (a single pass)** and storing only **compact mathematical summaries** in memory.

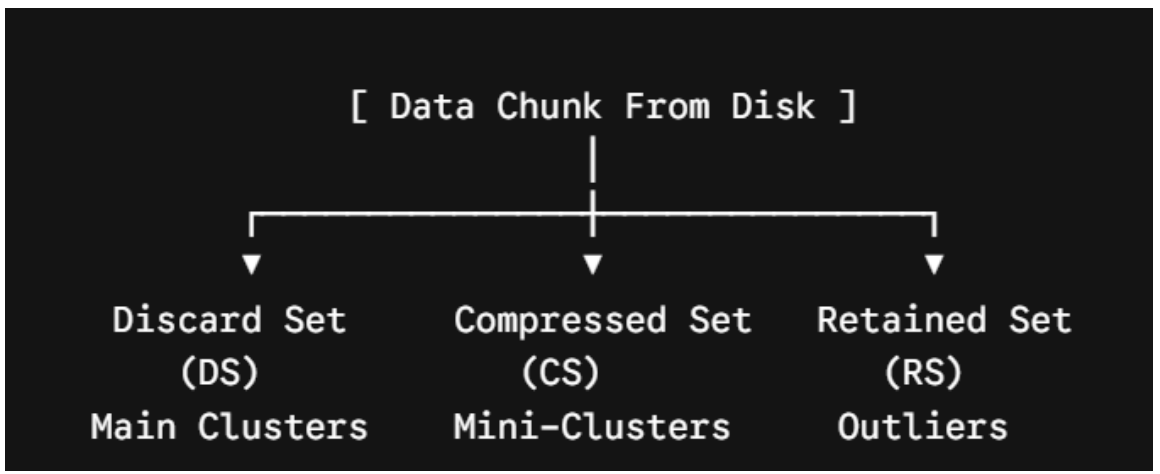
1. Core Assumptions of BFR

BFR makes two strict assumptions about the data:

1. **Clusters are Normally Distributed:** Data points in a cluster are normally distributed around a central mean in Euclidean space.
2. **Axis-Aligned Dimensions:** Attributes are independent within a cluster. This means clusters are **spherical or ellipsoidal** and aligned parallel to the coordinate axes (diagonal covariance matrix).

2. Memory Architecture: The Three Sets

As BFR reads data chunk-by-chunk from disk, it categorizes every point in memory into one of three distinct "sets":



A. Discard Set (DS)

- **What it stores:** Points that are confidently assigned to one of the k main clusters.
- **Key Rule:** Raw data points in the DS are **immediately discarded from memory** after updating the cluster's summary statistics.
- **Summary Storage:** Instead of keeping millions of points, BFR stores a tiny **Cluster Feature Vector**:
  - N: Total number of points in the cluster.
  - LS: Linear Sum vector of coordinates ($\sum_{i=1}^{N} x_i$) for each dimension.
  - SS: Squared Sum vector of coordinates ($\sum_{i=1}^{N} x_i^2$) for each dimension.

Memory Efficiency: A cluster with $10,000,000$ points in 100 dimensions requires storing only $1 + 100 + 100 = 201$ numbers!

B. Compressed Set (CS)

- **What it stores:** Groups of points that are close to each other but **not close enough** to any of the main k clusters to be merged into the DS.
- **Key Rule:** These form "mini-clusters". Like the DS, raw points in the CS are discarded, and only their summary vectors (N, LS, SS) are tracked.

C. Retained Set (RS)

- **What it stores:** True isolated points or outliers that do not belong to any main cluster (DS) or mini-cluster (CS).
- **Key Rule:** Raw coordinates of points in the RS **must be kept in memory** as individual points until future data chunks allow them to form a new mini-cluster or merge into an existing one.

3. How BFR Measures Distance (Mahalanobis Distance)

Standard k-means uses standard Euclidean distance, which assumes clusters are perfect round spheres of equal size. BFR uses **Mahalanobis Distance**, which normalizes the distance along each dimension using the variance (σ) of the cluster.

For a data point $x = (x_1, x_2, \ldots, x_d)$ and a cluster with mean $\mu = (\mu_1, \mu_2, \ldots, \mu_d)$ and standard deviation $\sigma = (\sigma_1, \sigma_2, \ldots, \sigma_d)$:

$$d_{\text{Mahalanobis}}(x, \mu) = \sqrt{\sum_{i=1}^{d} \left(\frac{x_i - \mu_i}{\sigma_i} \right)^2}$$

Computing μ and σ from Summaries (N, LS, SS):

Because BFR only stores (N, LS, SS), it calculates the mean and standard deviation on the fly:

- **Mean (μ_i):** $\mu_i = \frac{LS_i}{N}$
- **Variance (σ_i^2):** $\sigma_i^2 = \frac{SS_i}{N} - \left(\frac{LS_i}{N}\right)^2$

4. Step-by-Step Execution Algorithm

① Initialization

Seed initial centers

Load an initial chunk of data into memory. Run standard k-means (or k-means++) to pick k initial cluster centroids. Compute their initial (N, LS, SS) vectors to form the starting Discard Set (DS).

② Stream Data Chunks

Classify incoming points

Read a new chunk of data from disk into RAM. For each point x:

1. Measure its Mahalanobis distance to all k DS cluster centers.
2. If $d_{\text{Mahalanobis}}(x, \mu) < \text{Threshold}$ (e.g., $d < 3\sigma$), assign x to that DS cluster and update its (N, LS, SS).

③ Process Remaining Points

Form or expand CS and RS

For points not close enough to any main cluster:

1. Cluster them together with existing points in the Retained Set (RS).
2. If two or more RS points form a tight group, convert them into a new **Compressed Set (CS)** mini-cluster and store their (N, LS, SS).
3. Leftover isolated points remain in the **Retained Set (RS)** as raw points.

④ Merge Sets

Consolidate memory

Check if any CS mini-clusters can be merged together, or if any CS mini-clusters have moved close enough to be absorbed into a main DS cluster.

⑤ Final Wrap-up

End of stream assignment

Once all data chunks are processed, merge all remaining CS mini-clusters and RS outlier points into their nearest main DS cluster.

Summary Table: BFR vs. CURE vs. *k*-Means

Feature	Standard k-Means	BFR Algorithm	CURE Algorithm
Primary Use Case	Small/medium datasets	Massive datasets in a single pass	Non-spherical / complex geometries
Cluster Representation	1 Centroid	Summary Vectors (N, LS, SS)	c Shrunk Representative Points
Shape Assumption	Spherical	Spherical / Ellipsoidal	Any shape (curved, elongated)
Distance Metric	Euclidean Distance	Mahalanobis Distance	Min pair Euclidean Distance
Outlier Handling	Weak (distorts center)	Strong (retained in RS)	Strong (shrunk by factor alpha)